

Paper 2: Radius Sweep of Fully-Inscribed Unit-Cell Counts on a 4-Dimensional Lattice

An Enumeration Table from Radius 0.5 to 10.0 and a Reproducible Formulation

Author: Noriaki Kihara

Affiliation: WF System Co., Ltd.

ORCID: 0009-0004-6753-4020

Version: v0.2

Date: June 2026

DOI (this version): 10.5281/zenodo.20607574

Concept DOI: 10.5281/zenodo.20588038

License: CC BY 4.0

Abstract

In this paper we count the number of 4-dimensional unit cells of side 1, placed on a 4-dimensional integer lattice, that are fully inscribed in a 4-dimensional hyperball of radius R . The object is a purely geometric / integer-lattice counting problem, and we do not treat any correspondence with physical constants.

Taking the center of a unit cell to be

$$k = (k_1, k_2, k_3, k_4) \in \mathbb{Z}^4,$$

with half-width $1/2$ in each direction, the condition that this unit cell be fully inscribed in the hyperball of radius R is

$$\sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2 \leq R^2.$$

We define the number of lattice points satisfying this condition as

$$N_0(R) = \# \left\{ k \in \mathbb{Z}^4 \mid \sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2 \leq R^2 \right\}.$$

In this paper we compute $N_0(R)$ for radii $R = 0.5, 1.0, 1.5, \dots, 10.0$, and tabulate the diameter $2R$ of the circumscribing ball and the diagonal length $2\rho(R)$ of the actually stacked cell set.

1. Purpose

In Paper 1 we showed that choosing a 4-dimensional lattice in the wavelength space or the frequency space turns the sum-of-squares condition into a unit-cell counting problem.

In this paper we actually carry out that counting.

What we treat here is not a physical theory but the following purely geometric problem.

Among the unit 4-cells of side 1 on the 4-dimensional integer lattice, how many are fully inscribed in the 4-dimensional hyperball of radius R ?

2. Basic Definitions

Let the 4-dimensional integer lattice be

$$\mathbb{Z}^4.$$

To each lattice point

$$k = (k_1, k_2, k_3, k_4)$$

associate a 4-dimensional unit cell of side 1.

When the cell center is at k , each point x_i inside the cell satisfies

$$k_i - \frac{1}{2} \leq x_i \leq k_i + \frac{1}{2}.$$

The squared distance from the origin to the farthest vertex of this cell is

$$r_{\max}(k)^2 = \sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2.$$

Hence the condition that this cell be fully inscribed in the hyperball of radius R is

$$r_{\max}(k)^2 \leq R^2,$$

that is,

$$\sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2 \leq R^2.$$

3. Fully-Inscribed Cell Count

We define the fully-inscribed cell count as

$$N_0(R) = \# \left\{ k \in \mathbb{Z}^4 \mid \sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2 \leq R^2 \right\}.$$

Multiplying both sides by 4,

$$\sum_{i=1}^4 (2|k_i| + 1)^2 \leq (2R)^2.$$

Hence this problem can also be expressed as an integer inequality on a sum of four positive odd squares.

4. Stacked Diagonal Length

Let the set of cells inscribed at radius R be

$$K_R = \{ k \in \mathbb{Z}^4 \mid r_{\max}(k) \leq R \}.$$

Among the cells in this set, define the distance from the origin to the farthest vertex as

$$\rho(R) = \max_{k \in K_R} r_{\max}(k).$$

If K_R is empty, we set

$$\rho(R) = 0.$$

The centrally symmetric maximal diagonal length of the stacked cell set is then defined as

$$L(R) = 2\rho(R),$$

and the diameter of the circumscribing ball is

$$D(R) = 2R.$$

Hence always

$$L(R) \leq D(R).$$

5. Radius Sweep Results

5.1 Definitions of Volume, Filling Ratio, and Gap

In addition to the fully-inscribed cell count $N_0(R)$, we define the following quantities in order to evaluate the degree of filling relative to the surrounding hyperball.

The volume of the surrounding 4-dimensional hyperball of radius R is

$$V_0(R) = \frac{\pi^2}{2} R^4.$$

Since each fully-inscribed unit 4-cell has volume $1^4 = 1$, the total volume of the stacked cells is equal to the fully-inscribed cell count:

$$V_1(R) = N_0(R) \cdot 1^4 = N_0(R).$$

The filling ratio is therefore defined as

$$\text{fill}(R) = \frac{V_1(R)}{V_0(R)} = \frac{2 N_0(R)}{\pi^2 R^4}.$$

The filling ratio is always $\text{fill}(R) < 1.0$. Over a finite radius sweep it may show local increases and decreases, while for large R it tends toward 1.0.

The unfilled part of the surrounding hyperball volume is defined as the gap:

$$\text{gap}(R) = V_0(R) - V_1(R) = \frac{\pi^2}{2} R^4 - N_0(R) (> 0).$$

5.2 Sweep Results

Table 1 shows the results computed from radius $R = 0.5$ to 10.0 in steps of 0.5, with $R = 100, 1000, 10000$ also included. The values of $N_0(R)$, $2\rho(R)$, $\text{fill}(R)$, and $\text{gap}(R)$ are exact consequences of integer arithmetic, with the check values $N_0(1:5) = 1, 9, 137, 473, 1545$. The computational procedure is given in Appendix B.

Table 1. Radius sweep of fully-inscribed unit-cell counts on the 4-dimensional lattice, including filling ratio and gap

Circumscribed radius R	Diameter $2R$	Fully- inscribed cell count $N_0(R)$	Stacked diagonal $2\rho(R)$	Radial margin $R - \rho(R)$	Filling ratio $V1/V0$	Gap $V0-V1$
0.5	1	0	0	0.5	0.00000000	0.3
1	2	1	2	0	0.20264237	3.9
1.5	3	1	2	0.5	0.04002812	24.0
2	4	9	3.4641	0.267949	0.11398633	70.0
2.5	5	33	4.47214	0.263932	0.17119227	159.8
3	6	137	6	0	0.34274079	262.7
3.5	7	233	6.63325	0.183375	0.31464004	507.5
4	8	473	7.74597	0.127017	0.37441344	790.3
4.5	9	809	8.7178	0.141101	0.39978704	1,214.6
5	10	1545	10	0	0.50093193	1,539.3
5.5	11	2233	10.7703	0.114835	0.49450219	2,282.7
6	12	3457	11.8322	0.0839202	0.54053601	2,938.5
6.5	13	5001	12.8062	0.0968758	0.56771933	3,807.9
7	14	7281	14	0	0.61451024	4,567.5
7.5	15	9489	14.8324	0.0838015	0.60772296	6,125.0
8	16	12833	15.8745	0.0627461	0.63489001	7,379.9
8.5	17	16657	16.8523	0.0738502	0.64662328	9,103.0
9	18	22409	18	0	0.69212206	9,968.2
9.5	19	27233	18.868	0.0660189	0.67753435	12,961.3
10	20	34569	19.8997	0.0501256	0.70051440	14,779.0
100	200	476927289	199.99	0.005	0.96645675	16,552,931.1
1000	2000	4918072576937	2000	0	0.99660987	16,729,623,607.7
10000	20000	49331268889292600	20000	0	0.99966051	16,753,116,155,232.0

The filling ratio shows local increases and decreases, while for large R it tends toward 1.0. On the other hand, the gap corresponds to the boundary layer of the 4-dimensional hyperball. The leading scale of the boundary-layer volume is proportional to the surface area of the 4-dimensional ball. Therefore the leading order of the gap grows as R^3 . The asymptotic constant and the detailed lattice oscillations are left for separate analysis.

6. Basic Check Values

Extracting from Table 1 the first values at integer radius,

$$N_0(1) = 1,$$

$$N_0(2) = 9,$$

$$N_0(3) = 137,$$

$$N_0(4) = 473,$$

$$N_0(5) = 1545.$$

In particular,

$$N_0(3) = 137$$

is obtained as the number of unit 4-cells of side 1 fully inscribed in the 4-dimensional hyperball of radius 3.

This 137 is not introduced under any assumed correspondence with a physical constant; it is a pure counting result obtained from the 4-dimensional integer lattice, radius 3, the unit cell, and the full-inscription condition.

7. Decomposition at $R = 3$

For $R = 3$, the condition is

$$\sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2 \leq 9.$$

Multiplying both sides by 4,

$$\sum_{i=1}^4 (2|k_i| + 1)^2 \leq 36.$$

The possible shells of sums of odd squares in this range are

$$4, 12, 20, 28, 36.$$

The counts per shell are

$$4 : 1, \quad 12 : 8, \quad 20 : 24, \quad 28 : 40, \quad 36 : 64.$$

Hence

$$1 + 8 + 24 + 40 + 64 = 137.$$

Note that the shell 28 is the combined count of the $(3, 3, 3, 1)$ type and the $(5, 1, 1, 1)$ type.

8. Reproduction Algorithm

Table 1 can be reproduced by the following pseudocode.

```
def count_cells(R):
    count = 0
    max_s = 0
    max_abs = floor(R - 0.5) + 1

    for k1 in range(-max_abs, max_abs + 1):
        for k2 in range(-max_abs, max_abs + 1):
            for k3 in range(-max_abs, max_abs + 1):
                for k4 in range(-max_abs, max_abs + 1):
                    s = sum((abs(ki) + 0.5)**2 for ki in [k1, k2, k3, k4])
                    if s <= R**2:
                        count += 1
                        max_s = max(max_s, s)

    rho = sqrt(max_s) if count > 0 else 0
    diagonal = 2 * rho
    return count, rho, diagonal
```

This computation merely judges, for each cell, whether its farthest vertex lies within the ball of radius R .

9. On the Absence of Uncertainty

This paper does not treat the weighted contribution of near-boundary cells or the effective counting including the uncertainty width δ .

That is, what is computed in this paper is the fully-inscribed counting at

$$\delta = 0.$$

To perform counting with uncertainty, the indicator function must be replaced by a weight function

$$W_\delta(x).$$

However, how to determine the value of δ or the form of that function is undefined and outside the scope of this paper.

10. Conclusion

In this paper we computed, from radius 0.5 to 10.0 in steps of 0.5, the number of unit 4-cells of side 1 on the 4-dimensional integer lattice that are fully inscribed in the 4-dimensional hyperball of radius R .

The fully-inscribed cell count is defined by

$$N_0(R) = \# \left\{ k \in \mathbb{Z}^4 \mid \sum_{i=1}^4 \left(|k_i| + \frac{1}{2} \right)^2 \leq R^2 \right\}.$$

This counting is purely geometric / integer-lattice in nature and asserts no correspondence with physical constants.

In particular,

$$N_0(1) = 1, \quad N_0(2) = 9, \quad N_0(3) = 137$$

are obtained.

The results of this paper provide a basic reference showing that the sum-of-squares constraint, when the wavelength space or the frequency space is chosen as a 4-dimensional lattice, can be treated as a concrete unit-cell counting problem.

Appendix A: CSV File

Table 1 of this paper is also saved as the accompanying file `paper2_radius_sweep_R_0_5_to_10_step_0_5.csv`.

The meaning of the columns is as follows.

- `R`: radius of the circumscribing ball
 - `sphere_diameter_2R`: diameter $2R$ of the circumscribing ball
 - `full_inclusion_cell_count_N0`: fully-inscribed cell count $N_0(R)$
 - `stack_vertex_radius`: maximal vertex radius $\rho(R)$ of the stacked cell set
 - `stack_diagonal_length`: stacked diagonal length $2\rho(R)$
 - `unused_margin_R_minus_stack_radius`: radial margin $R - \rho(R)$
 - `shells_S_counts`: count per shell $S = \sum_i (2|k_i| + 1)^2$
-

Appendix B: Filling-Ratio and Gap Calculation Program

The values of $N_0(R)$, $2\rho(R)$, the filling ratio V_1/V_0 , and the gap $V_0 - V_1$ in Table 1 can be reproduced exactly by the accompanying Python script

`paper2_count_fill_gap.py`.

The naive quadruple loop in Section 8 has computational complexity $O(R^4)$ and becomes impractical for $R = 100$ and above. Therefore the script rewrites the full-inscription condition as the integerized positive odd-square condition

$$\sum_{i=1}^4 (2|k_i| + 1)^2 \leq (2R)^2,$$

constructs the one-axis odd-square distribution, convolves it into a two-axis distribution g_2 , and then evaluates $N_0(R)$ by cumulative sums in roughly $O(R^2)$. The value of $2\rho(R)$ is obtained by a two-pointer search over the keys of g_2 . All computations are performed using integer arithmetic; the values up to $R = 10000$ are exact rather than approximate. As a checksum, the script internally confirms $N_0(1:5) = 1, 9, 137, 473, 1545$.